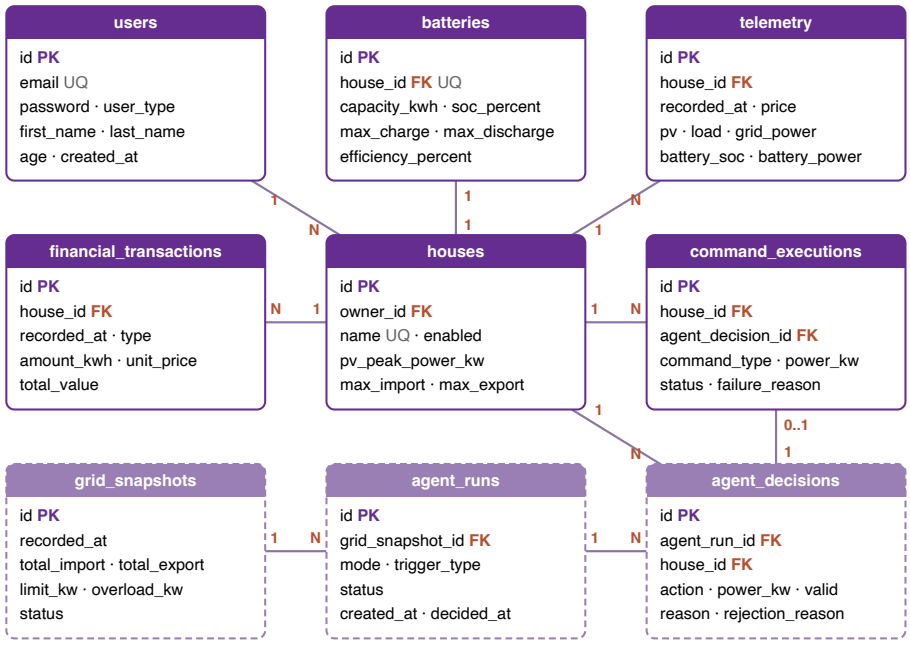
Structura bazei de date

MySQL 8.4 · Flyway · 9 tabele, toate în `solar_db` a API-ului · se construiesc feature cu feature, după `PLAN.pdf` · Sebi

Diagrama



PK cheie primară, `BIGINT` auto-increment · **FK** cheie străină · **UQ** valoare unică · **NN** obligatoriu (`NOT NULL`) · chenar punctat = tabel nou (F8 și F10) · eticheta de lângă nume = feature-ul și migrarea care creează tabelul · **coloană cărămizie** = nouă sau schimbată față de codul tău

Tabelele

users F1 · V1 · F16 · V8

Contul: proprietar (`USER`), `OPERATOR`, `ADMIN` sau agentul.

coloană	tip	reguli
id	BIGINT	PK
first_name	VARCHAR(100)	NN
last_name	VARCHAR(100)	NN
age	INT	NN, ≥ 18
email	VARCHAR(100)	NN, UQ
password	VARCHAR(100)	hash BCrypt · **dispare în V8**
keycloak_id	VARCHAR(36)	UQ · sub din token · V8
user_type	VARCHAR(20)	NN · ADMIN OPERATOR USER AGENT
created_at	DATETIME(6)	audit (@CreatedDate) · V1

Permisiunile rezultă din `user_type`, dintr-o singură hartă în cod: `user_permissions` dispare. `ADMIN` vine din `.env` la pornire; `AGENT` îl creează administratorul (F10).

houses F1 · V1 · F2 · V2

O casă a comunității, cu panourile și limitele ei de rețea.

coloană	tip	reguli
id	BIGINT	PK
owner_id	BIGINT	NN, FK → users.id
name	VARCHAR(100)	NN, UQ · V2
pv_peak_power_kw	DOUBLE	NN, > 0
max_import_power_kw	DOUBLE	NN, > 0
max_export_power_kw	DOUBLE	NN, > 0
enabled	BOOLEAN	NN, implicit true
created_at	DATETIME(6)	audit · V2

`houses.battery_id` există în V1 (cheie străină spre baterie, în plus față de `batteries.house_id`) și iese în V2: legătura rămâne într-un singur sens.

batteries F1 · V1 · F2 · V2

Bateria casei. Se creează odată cu casa (o casă = o baterie).

coloană	tip	reguli
id	BIGINT	PK
house_id	BIGINT	NN, UQ, FK → houses.id
capacity_kwh	DOUBLE	NN, > 0 · V2
soc_percent	DOUBLE	NN, între 10 și 90
max_charge_power_kw	DOUBLE	NN, > 0
max_discharge_power_kw	DOUBLE	NN, > 0
efficiency_percent	DOUBLE	NN, (0, 100] · V2, din efficienty_percent

command_executions F1 · V1 · F3 · V3 · F10 · V7

Fiecare comandă trimisă unei baterii: manuală sau a agentului.

coloană	tip	reguli
id	BIGINT	PK
house_id	BIGINT	NN, FK → houses.id
agent_decision_id	BIGINT	FK, gol la comenzile manuale · V7
command_type	VARCHAR(20)	NN · CHARGE DISCHARGE IDLE
power_kw	DOUBLE	NN, ≥ 0
status	VARCHAR(20)	NN · SUCCESS REJECTED
failure_reason	VARCHAR(50)	motivul, la REJECTED · V3 (lungime)
soc_after_percent	DOUBLE	NN · V3 (în V1 poate fi gol)
executed_by	VARCHAR(255)	NN · emailul; la agent: agentul + cine a aprobat
created_at	DATETIME(6)	NN

V3 adaugă un index pe `(house_id, created_at)`: istoricul unei case, cele mai noi primele, nu citește tot tabelul.

telemetry F1 · V1 · F4 · V4

Ce s-a măsurat la o casă, la un tick al simulării.

coloană	tip	reguli
id	BIGINT	PK
house_id	BIGINT	NN, FK → houses.id
recorded_at	DATETIME(6)	NN · ora virtuală · V4, din local_date_time
pv_power_kw	DOUBLE	NN, ≥ 0
load_power_kw	DOUBLE	NN, ≥ 0
grid_power_kw	DOUBLE	NN · + export, – import
battery_soc_percent	DOUBLE	NN
battery_power_kw	DOUBLE	NN · + descărcare
price_per_kwh	DECIMAL(10,4)	NN · lei/kWh · V4

UQ pe `(house_id, recorded_at)`: un rând pe casă și oră, iar indexul lui face ca „ultimele 50” să nu citească tot tabelul.

financial_transactions F1 · V1 · F5 · V5

La fiecare tick: importul casei = `BUY`, exportul = `SELL`.

coloană	tip	reguli
id	BIGINT	PK
house_id	BIGINT	NN, FK → houses.id · V5
recorded_at	DATETIME(6)	NN · ora tick-ului · V5
transaction_type	VARCHAR(10)	NN · BUY SELL
amount_kwh	DOUBLE	NN, > 0
unit_price	DECIMAL(10,4)	NN · V5
total_value	DECIMAL(12,2)	NN, kWh × preț · V5

Banii pe `BigDecimal`: în `double`, `0.1 + 0.2 ≠ 0.3`. UQ pe `(house_id, recorded_at)`: același tick primit de două ori nu dublează banii (F13).

grid_snapshots F8 · V6

Transformatorul comun, la fiecare tick: un rând pentru toată comunitatea.

coloană	tip	reguli
id	BIGINT	PK
recorded_at	DATETIME	NN, UQ
total_import_kw	DOUBLE	NN, ≥ 0
total_export_kw	DOUBLE	NN, ≥ 0
limit_kw	DOUBLE	NN · 50 – 10% = 45
overload_kw	DOUBLE	NN, ≥ 0
status	VARCHAR(20)	NN · OK WARNING CRITICAL

Peste 45 kW = `WARNING`, peste 50 = `CRITICAL`; `overload_kw` = cât trece peste `limit_kw`. Importul și exportul se verifică separat.

agent_runs F10 · V7

Un plan propus de agent prin MCP: ce stare a văzut, cine l-a făcut și ce s-a întâmplat cu el.

coloană	tip	reguli
id	BIGINT	PK
grid_snapshot_id	BIGINT	NN, FK
mode	VARCHAR(20)	NN · AgentMode la momentul propunerii
proposed_by	VARCHAR(100)	NN · emailul contului AGENT
planner	VARCHAR(20)	NN · RULES (F11) LLM (F12)
trigger_type	VARCHAR(20)	NN · MANUAL SCHEDULED EVENT (F13)
planning_ms	INT	cât a durat planificarea
status	VARCHAR(20)	NN · PROPOSED EXECUTED REJECTED
created_at	DATETIME	NN
decided_by	VARCHAR(100)	cine a hotărât
decided_at	DATETIME	

agent_decisions F10 · V7

O comandă propusă de agent pentru o casă, cu motivul ei.

coloană	tip	reguli
id	BIGINT	PK
agent_run_id	BIGINT	NN, FK → agent_runs.id
house_id	BIGINT	NN, FK → houses.id
action	VARCHAR(20)	NN
power_kw	DOUBLE	NN, ≥ 0
reason	VARCHAR(500)	NN · „surplus PV”; la LLM, mai lung
valid	BOOLEAN	NN · verdictul validatorului
rejection_reason	VARCHAR(50)	când `valid = false`

UQ pe `(agent_run_id, house_id)`: o decizie pe casă într-o rulare. `planner` și `planning_ms` sunt datele comparației reguli vs. AI (F12).

Migrările, feature cu feature. De la F1, schema o face Flyway, iar `ddl-auto: validate` verifică la fiecare pornire că tabelele se potrivesc cu entitățile. Deci o migrare intră **în același commit** cu schimbarea entității, iar una care a rulat nu se mai editează. Toate cinci (V1–V5) au fost aplicate la rând pe un MySQL 8.4 gol, cu entitățile de la finalul F5 și `validate`: pornirea trece.

migrare	feature	ce face
`V1__schema`	F1	cele 6 tabele **exact cum le creează Hibernate din entitățile de la finalul F1**, cu numele cu greșeli rămase (`bateries`, `efficienty_percent`, `financial_transaction`, `timestamp`, `local_date_time`) și cu constrângeri cu nume (`fk_…`, `uq_…`). `users.created_at` intră (audit); `user_permissions` nu mai există (permisiunile vin dintr-o hartă în cod); `agent` și `agent_seq` din fișierul vechi nu au entitate și nu intră. `houses.battery_id` ↔ `bateries.house_id`: două chei străine una spre alta, create cu `alter table` după ambele tabele; `user_type` e `VARCHAR(20)` (Hibernate generează `enum`, iar `validate` acceptă ambele)
`V2__house_battery`	F2	`houses`: iese `battery_id` (cheia străină și indexul unic după nume, apoi coloana), intră `created_at` și un unic pe `name` · `bateries` → `batteries` · `efficienty_percent` → `efficiency_percent` · `capacity_kwh` `NOT NULL` (10.0 pe rândurile existente; azi 10.0 e scris de două ori, în serviciu și în validator)
`V3__commands`	F3	`command_type` și `status` `VARCHAR(20)`, `failure_reason` `VARCHAR(50)`, `soc_after_percent` `NOT NULL`, index pe `(house_id, created_at)` pentru istoric (`power_kw`, `failure_reason`, `soc_after_percent` sunt deja în V1: entitatea le are)
`V4__telemetry`	F4	`local_date_time` → `recorded_at`, `price_per_kwh` `DECIMAL(10,4) NOT NULL` (cu `DEFAULT` temporar), unic pe `(house_id, recorded_at)`
`V5__financial`	F5	`financial_transaction` → `financial_transactions`, `timestamp` → `recorded_at`, toate coloanele `NOT NULL`, bani pe `DECIMAL`, cheie străină pe `house_id`, unic pe casă + oră
`V6__grid_snapshots`	F8	tabelul nou al transformatorului
`V7__agent`	F10	`agent_runs`, `agent_decisions`, `command_executions.agent_decision_id`
`V8__users_keycloak`	F16	`password` iese, `keycloak_id` intră: conturile le ține Keycloak, API-ul păstrează doar proprietarul local

Fără migrare: F6 (teste pe MySQL real) și F7 (loguri). Fără tabele: agentul (F11) și simulatorul (F14) n-au bază de date. Agentul vede totul prin MCP, simulatorul doar publică în Kafka. Pe EC2, V1 nu rulează peste o schemă făcută de Hibernate: volumul `mysql_data` se șterge o singură dată, înainte de F1 (datele sunt generate).